7. Tasarım Desenleri

Yapılar inşaa etmek insanoğlunun binlerce yıldır tekrarladığı bir iştir. İnsan, yerleşik düzenin temelini oluşturan barınak yapımından günümüzün gökdelenlerine kadar her şeyi inşa etmiş ve bunun tecrübesini nesilden nesile aktarmıştır. Neticede, günümüzde insanoğlu binlerce yıllık bir tecrübeyi kullanarak daha büyük ve daha gelişmiş yapılar inşa etmektedir. Burada anahtar kavramlar tecrübe aktarımı ve bunun çalışma zamanındaki karşılığı tekrarlamadır.

Pragmatik olarak tecrübe aktarma ve tekrarlama kavramlarını tasarım desenleri ve bunun çalışma zamanı karşılığı tekrar kullanılabilirlik (re-usability) ile gösterebiliriz. Tasarım probleminin çözümüne yönelik tasarım desenlerini kullanmak ve bu tasarımları içeren kod bileşenlerini tekrar kullanmak kurumsal uygulama geliştirme projesinin başarıya ulaşması için gerekli pratiklerdendir.

7.1. Tasarım Desenleri (Design Pattern)

Tasarım desenleri, bir yazılım problemini en basit ve en efektif bir şekilde çözüme kavuşturacak yöntemler topluluğudur. "Design pattern" kavramı bir kurallar topluluğundan ziyade bir işi nasıl ve en güzel ne şekilde yapabileceğimizi gösteren yöntemler topluluğudur.

Yapılar inşaa etmek insanoğlunun binlerce yıldır tekrarladığı bir iştir. İnsan, yerleşik düzenin temelini oluşturan barınak yapımından günümüzün gökdelenlerine kadar her şeyi inşa etmiş ve bunun tecrübesini nesilden nesile aktarmıştır. Neticede, günümüzde insanoğlu binlerce yıllık bir tecrübeyi kullanarak daha büyük ve daha gelişmiş yapılar inşa etmektedir. Burada anahtar kavramlar tecrübe aktarımı ve bunun çalışma zamanındaki karşılığı tekrarlamadır. Benzer şekilde Tasarım Desenleri kavramı mimari bir kavram olduğunu belirtilmiştir[Alexander].

Design is not just what it looks like and feels like. ***Design is how it works.***

Steve Jobs

Günümüzde yazılım mühendisliği alanında en fazla ses getiren kurgu yazılımın gerçek dünya ile olan ilişkisinin sağlanabilmesidir. Bu ilişki elbette nesne yönelimli programlama tekniği ile sağlanmaktadır. Nesne yönelimli programlama tekniği bilgisayar uygulamalarını soyut bir olgudan çıkararak insanların daha kolay algılayabileceği hale getirmiştir. Öyle görünüyor ki, makro düzeyde gerçek hayatı modelleme ile başlayan bu ilişki mikro düzeyde de büyük ses getirecektir. Nitekim son yıllarda geliştirilen yapay sinir ağları ile makinelerin çalışma sistemlerinin gün geçtikçe insanların veya canlıların çalışma şekline yaklaştığı görülmektedir. Artık bilgisayarlardan sadece verilen komutları yerine getirmek değil, belirli olaylar ve durumlar karşısında bazı hükümlere varabilmeleri de istenmektedir. Yazılım mühendisleri veya programcılar olarak gerçek hayatı ne kadar iyi modelleyebiliyorsa o kadar başarılı sayılı. O halde "tasarım desenleri" konusu nerede devreye girmektedir? Tasarım desenleri bir modelin tasarımın ustaca tasarlanmasını sağlayacak çeşitli desenlerin oluşturulmasını ve bu desenlerin ihtiyaç dahilinde herhangi bir modelin inşasında kullanılabilmesini sağlar. Tasarım desenleri kavramı bir kurallar topluluğundan ziyade bir işi nasıl ve en güzel ne şekilde yapabileceğimiz gösteren yöntemler topluluğudur. Öyleki iyi bir yazılım modelleyicisiyseniz kendi tasarım desenlerinizi oluşturabilir ve bunları diğer ustaların kullanımına sunabilirsiniz. Tasarım desenleri tecrübe ile oluşturulan yapılardır. Bazıları olmazsa olmaz yapılar olmasına rağmen bazıları tamamen yazılımın sanatsal yönünü göstermek için tasarlanmıştır.

Great design will not sell an inferior product, but it will enable a great product to achieve its maximum potential.

Thomas Watson Jr

Pragmatik olarak tecrübe aktarma ve tekrarlama kavramlarını tasarım desenleri ve bunun çalışma zamanı karşılığı tekrar kullanılabilirlik (re-usability) ile gösterebiliriz. Tasarım probleminin çözümüne yönelik tasarım desenlerini kullanmak ve bu tasarımları içeren kod bileşenlerini tekrar kullanmak kurumsal uygulama geliştirme projesinin başarıya ulaşması için gerekli pratiklerdendir.

Tasarım Desenleri (diğer adlarıyla "Tasarım Kalıpları", "Tasarım Şablonları", "Tasarım Motifleri" "Design Pattern") birbirine benzer sorunlar karşısında denenmiş, test edilmiş ve işlerliğini kanıtlamış çözümler sunarken yazılımcıların tecrübeleri doğrultusunda oluşturulurlar. Buradan hareketle Tasarım Desenleri;

- Nesne temelli tasarım
- Soruna en verimli & denenmiş çözüm yöntemi
- Tekrarlanan sorunlara tekrarlanan çözümler
- Kolay okunabilirlik & Ortak dil

çözümleri sunarlar.

7.2. Dörtlü Çete (GoF) Tasarım Desenleri

- **Yaratım Desenleri** (Creational Patterns)

  Singleton, Abstract Factory, Factory Method, Builder, Prototype

- **Yapısal Desenler** (Structural Patterns)

  Adapter, Bridge, Composite, Decorater, Facade, Flyweight, Proxy

- Davranışsal Desenler (Behavioral Patterns)

  CoR, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

7.5.1. Yaratım Desenleri

Yaratım tasarım desenleri, yazılım nesnelerinin (ya da başka bir değişle sınıf örnekleri - class instances) nasıl yaratılacağı hakkında öneriler sunar. Anafikir, iyi bir yazılımın, içinde barındırdığı nesnelerin nasıl yaratıldığından bağımsız olarak tasarlanması gerekliliğidir. Diğer bir deyişle, nesnelerin nereden ve nasıl yaratıldığı, ait oldukları yazılımın işleyişini etkilememeli; yeni özellikler eklenmesine ve değişikliklere karşı sorun oluşturmamalıdır.

Yazılım sistemleri geliştikçe, nesnel bileşimler (object composition), sınıf kalıtımına (class inheritence) göre daha fazla önem kazanır. Bunun nedeni, yazılım sistemleri için basit temel davranış (behavior) şekillerinin tanımlanması üzerine kurulu tasarımların, sabit davranışlara dayalı tasarımlara göre daha esnek olmasındandır. Diğer bir deyişle, nesnelere davranışların bileşim olarak eklenmesi, daha sonra bu davranışların yazılımın gelişimine göre değiştirilmesine olanak sağlar. Bu durumda, geliştirilen yazılım için gereken temel davranış şekillerine dayalı bir tasarım, nesne arayüzleri (interface) değiştirmeden farklı ya da daha karmaşık davranışların kullanılabilmesini olanaklı kılar.

Ancak, nesnel bileşimler yoluyla temel davranışları sağlayan nesnelerin örneklenmesi, ana ya da kalıtım yoluyla davranış değişikliğine uğratılarak türetilmiş sınıflardan nesne oluşturmak daha

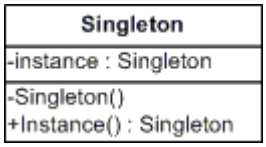
zordur. Yaratım tasarım desenleri bu zorlukları aşmak amacıyla kullanılabilecek yazılım desenlerini içerir.

Yaratım tasarım desenleri, hem hangi somut sınıfların (concrete class) nesne örneklemesinde kullanıldığını, hem de bu örneklerin nasıl yaratılıp bir araya getirildiğini yazılım sisteminden saklarlar.

7.5.1.1. Singleton Tasarım Deseni

Bir sınıftan sadece bir tane nesne yaratılacak şekilde kısıtlama sağlar. Söz konusu nesneye uygulamanın her yerinden ulaşılabilir. Nesne ilk kez kullanılana dek yaratılmayabilir.

UML Sınıf Diyagramı:



Şekil 1: Singleton Tasarım Deseni UML Class Diagramı

Java Kodları

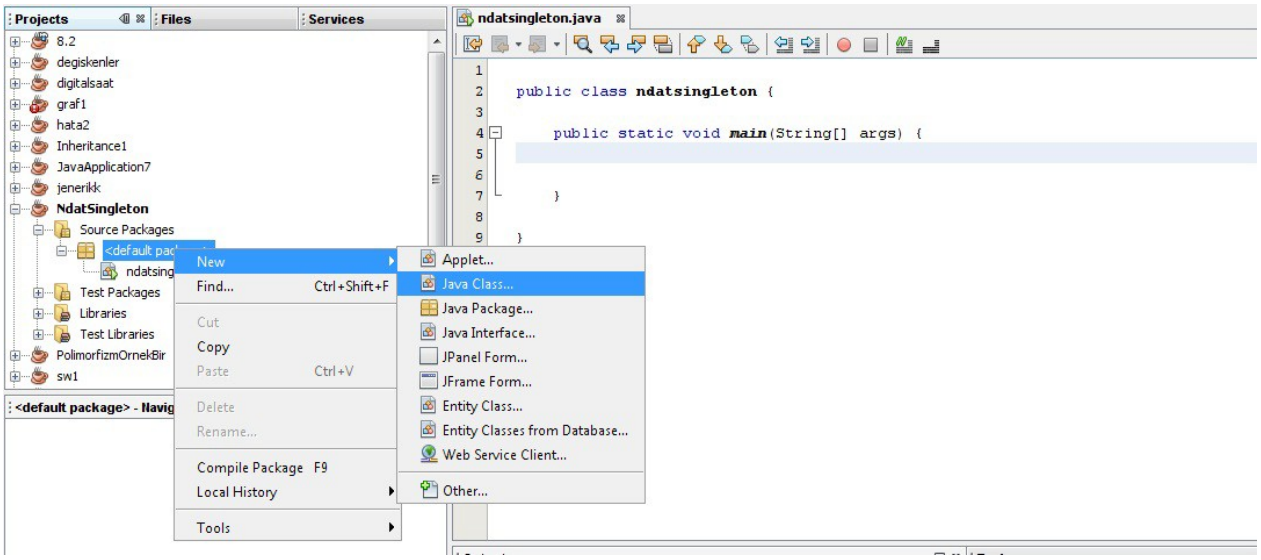
```
public class singletonSinif {
//2. adım: Kendi sınıfımızdan bir global değişken oluşturalım
// Daha sonra yazacağımız metotta bu değişkeninin oluşturulup
//oluşturulmadığını konrol edeceğiz.
   private static singletonSinif sinifOrnegi;
   //1. adım: öncelikle kurucumuzu private erişim denetleyicisi durumuna indirgeyelim.
   private singletonSinif()
   {
      //Boş Kurucu
   }
// 3. Adım: Temebel ilklendirme (lazy initialization)
    public static singletonSinif TekNesneOlustur ()
    {
       if (sinifOrnegi==null)
       {
```

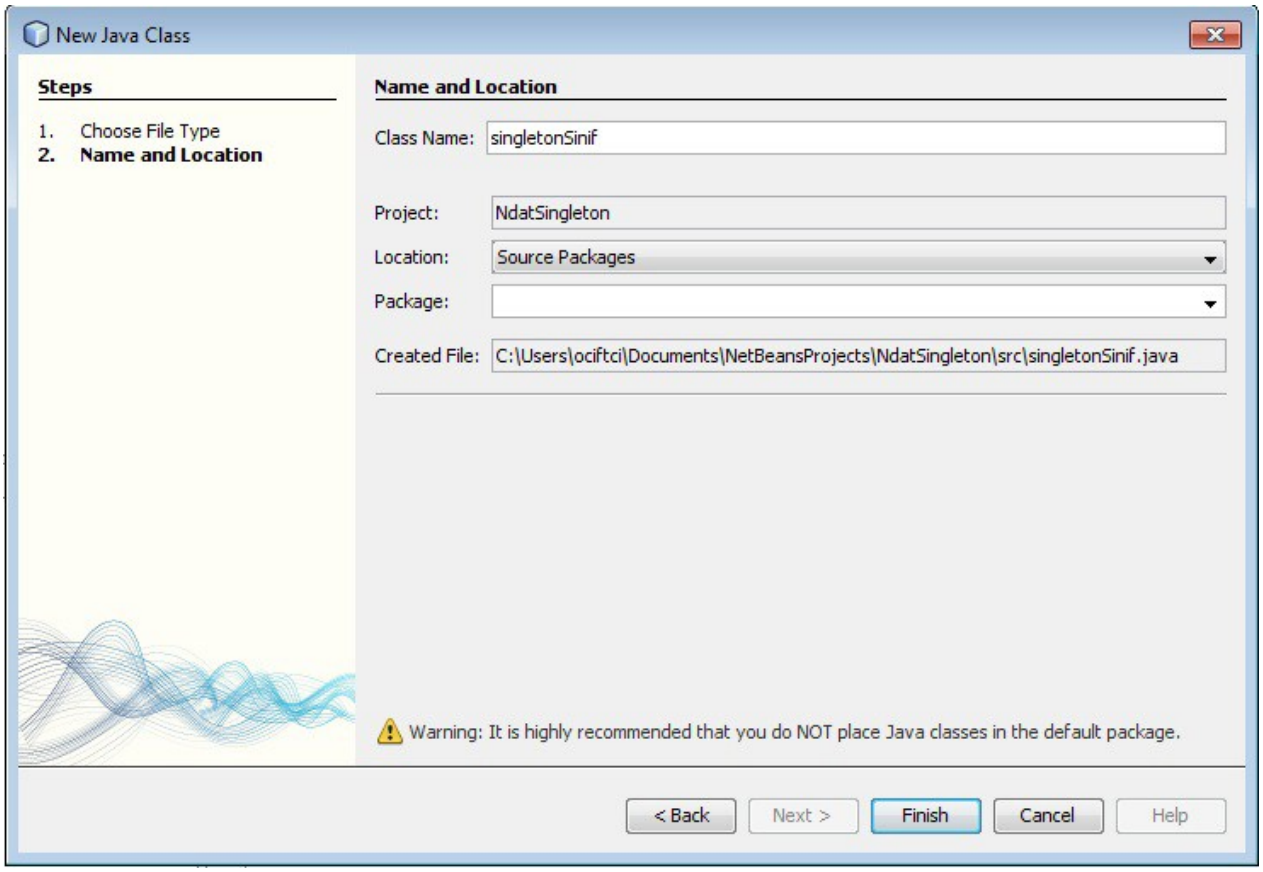
```
        //4. Adım:
        //Aynı anda erişilmek istenildiğinde
        //prosesi kitlemek gerekmektedir
       synchronized (singletonSinif.class)
       {
        if (sinifOrnegi==null)
            sinifOrnegi = new singletonSinif();
       }
      }
      return sinifOrnegi;
   }

   //5. Adım Ekrana yazdırma işlemi.
     public void Yazdir()
     {
       System.out.println(this);
     }
 }

public class ndatsingleton {
   public static void main(String[] args) {
     singletonSinif tekNesne=singletonSinif.TekNesneOlustur();
    /* if (tekNesne != null)
       System.out.println(tekNesne);
    else
       System.out.println("Nesnemiz oluşturulmadı.");*/
     //5. Adım denemesi
     for(int i =0; i < 5; i ++)
     {
          tekNesne.Yazdir();
     }
   }
}
```

Kod Açıklamaları.





Açtığımızı Java projesine öncelikle yeni bir sınıf ekliyoruz. Sınıfımızın adı ***singletonSinif*** olsun.

```
ndatsingleton.java   singletonSinif.java

1   public class singletonSinif {
2
3
4
5       //öncelikle kurucumuzu private erişim denetleyicisi durumuna indirgeyelim.
6
7       private singletonSinif(){
8       }
9       }
10
11  }
12
```

İlk etapta yapmamız gereken; sınıfımızı ***new*** anahtar kelimesiyle oluşturulabilmesini engellemek amacıyla sınıf kurucusunun erişimini ***private*** yapmak.

```
public class singletonSinif {

//2. adım: Kendi sınıfımızdan bir global değişken oluşturalım
// Daha sonra yazacağımız metotta bu değişkeninin oluşturulup
//oluşturulmadığını konrol edeceğiz.
    private static singletonSinif sinifOrnegi=null;

    //1. adım: öncelikle kurucumuzu private erişim denetleyicisi durumuna indirgeyelim.
```

2. adımda sınıf türünden bir değişken oluşturulur. Burada *sinifOrnegi* isimli değişken her ne ne kadar **null** değere sahip olsa da yinede başlangıç değerinin atamasını yapmakta yarar vardır.

```
    }


// 3. Adım: Temebel ilklendirme (lazy initialization)
    public static singletonSinif TekNesneOlustur ()
    {
        if (sinifOrnegi==null)
        {
            //4. Adım:
            //Aynı anda erişilmek istenildiğinde
            //prosesi kitlemek gerekmektedir
          synchronized (singletonSinif.class)
          {
            if (sinifOrnegi==null)
                sinifOrnegi = new singletonSinif();

          }
        }
        return sinifOrnegi;
    }
}
```

Sınıfımıza **TekNesneOlustur** isimli bir metod ekliyoruz ve 2. Adımda oluşturduğumuz değişkenin **null** olup olmadığını kontrol ediyoruz. Eğer sınıf daha önceden hafızada oluşturulmuş ise bu değer =**null** olmayacağından direk olarak son satırda ki **return** ifadesine dönecektir. **null** olması durumunda ise aynı anda birden fazla isteğin olabilme ihtimaline karşılık sınıfı kanalını kilitliyoruz(multi threaded). Böylelikle işlemin biri sürerken diğerleri işlem bitene kadar bekleyeceklerdir. (Bu tekniğe double checked locking denmektedir.)

```
1
2   public class ndatsingleton {
3
4       public static void main(String[] args) {
5
6           singletonSinif tekNesne=singletonSinif.TekNesneOlustur();
7
8           if (tekNesne != null)
9               System.out.println(tekNesne);
10          else
11              System.out.println("Nesnemiz oluşturulmadı.");
12
13      }
14
15  }
16
```

```
Output - NdatSingleton (run)                    Tasks
run:
singletonSinif@9304b1
BUILD SUCCESSFUL (total time: 1 second)
```

main metodu içersinde ilk etapta sınıfımızın çalışıp çalışmadığını kontrolünü yaparak ekranda sonucun görünmesini sağlıyoruz.

Son adımda ise ndatsingleton isimli sınıfımıza yazdırmayla ilgili aşağıdaki kodlarımızı ekliyoruz.

```
//5. Adım Ekrana yazdırma işlemi.
public void Yazdir()
{
    System.out.println(this);
}
```

main içersinde dileğimiz kadar

```
1
2   public class ndatsingleton {
3
4       public static void main(String[] args) {
5
7
8           /*  if (tekNesne != null)
9               System.out.println(tekNesne);
10          else
11              System.out.println("Nesnemiz oluşturulmadı.");*/
12
13          //5. Adım denemesi
14          for(int i =0; i < 5; i ++)
15          {
16              singletonSinif tekNesne=singletonSinif.TekNesneOlustur();
17              tekNesne.Yazdir();
18          }
19
20      }
21
22  }
23
```

Variables | Breakpoints | Output | Tasks

NdatSingleton (debug) | Debugger Console | NdatSingleton (run)

```
run:
singletonSinif@9304b1
singletonSinif@9304b1
singletonSinif@9304b1
singletonSinif@9304b1
singletonSinif@9304b1
BUILD SUCCESSFUL (total time: 0 seconds)
```

Sınıfımızdan kaç tane nesne oluşturulursa oluşturulsun hep aynı nesneyi çağırmaktadır.

Çalışma sorusu: Bir sınıfın tek olması hangi durumlarda gerekebilir?